Web Systems Development

Security: is it a feature you add, or a property of how the system was designed?

Session 19 · Tue 11/3 · the OWASP Top 10, and six holes in one to-do app

RPI

TODAY

A map, six holes, and a lab on Friday

01 **The map**

The OWASP Top 10:2025 — the field's shared list of what goes wrong, and what changed this year.

02 **The finished product**

Every app you built this term had these holes closed for you. We find where, and why.

03 **Six holes, one to-do app**

Injection, XSS, CSRF, a leaked secret, a bad dependency, missing headers — each as an attack and a one-line fix. Friday's six drills, in order.

04 **One idea**

Three design principles from 1975 that every fix today is an instance of.

05 **Friday: you fix them**

The security lab, with a Check-result button. Homework 7 due; Homework 8 assigned: the same six, on your own api/.

BUILDS ON

Where this sits: code you have already written

YOU ALREADY DID

week 3

prepare() and ? placeholders in PHP

week 7

Blade: {{ }} escapes by default; @csrf

week 9

Eloquent: where() binds the value

HW 7

your team's api/: tables, models, five routes

TODAY

each of those

seen as the attack it was preventing

plus three

you have not met: secrets, dependencies, headers

the map

OWASP's ten names for what goes wrong

one idea

assume every input is hostile; fail closed

NEXT

Fri 11/6

the lab: six attacks, six fixes, Check result · HW 7 due

HW 8

the same six on your own api/

Tue 11/10

authentication: logging in, sessions vs. tokens, SSO

HW 9

the login and the write endpoints

One running example all day: the to-do list from Homeworks 6 and 7, which is Friday's lab. Where a hole touches logging in, the pantry example stands in for your project.

You have been secure all term — by default

Blade escaped your output. Eloquent bound your queries. .env stayed out of git. Laravel checks a CSRF token on every POST.

You never saw any of it, because the defaults are the security. Today we turn them off, and watch.

What the framework already does for you

OUTPUT

`{{ $x }}`

Blade escapes every value, so a <script> in your data shows as text. (week 7, drill 4)

QUERIES

`where('a', $v)`

Eloquent sends the value bound, separate from the SQL, so it can't be injected. (week 9; week 3's prepare())

FORMS

`@csrf`

Every POST must carry a token Laravel checks, so another site can't post as you. (this week's reading; your project's api.js does it by hand)

SECRETS

`.env`

.gitignore keeps your keys out of the repository without your doing anything. (every Laravel app since week 7)

The field's shared list of what goes wrong

A01	**Broken Access Control**	drill 3 · next week	A02	**Security Misconfiguration**	drills 4, 6
A03	**Software Supply Chain Failures**	drill 5	A04	Cryptographic Failures	
A05	**Injection**	drills 1, 2	A06	Insecure Design	
A07	**Authentication Failures**	next week	A08	Software or Data Integrity Failures	
A09	Security Logging & Alerting Failures		A10	Mishandling of Exceptional Conditions	

Three entries moved. Why?

PREDICT In 2021 Injection was #3 and the list had no "supply chain" entry at all. In 2025:

```
Injection              #3  (2021)   ->   #5  (2025)   fell
Security Misconfig.    #5  (2021)   ->   #2  (2025)   rose
Software Supply Chain   —  (new)    ->   #3  (2025)   new
```

Predict with a partner: did injection get rarer, or did the frameworks get better? And why would "someone else's code" become a top-three risk in four years?

Injection

A05:2025. Your input is text. Injection is when that text is read as a command instead — SQL in the database, HTML in the browser.

Drills 1 and 2 on Friday.

What does this search box return?

PREDICT The to-do search, as the lab's starter app has it. A visitor types the second line into the box.

```
$sql = "select id, title, done from todos where title like '%$q%' order by title";

the search box:    ' UNION SELECT id, email, 0 FROM users WHERE email LIKE '%
```

Predict with a partner: write out the SQL the database actually receives. What table does it read? (Hint: the typed quote closes the like-string, and UNION glues a second query onto the first.)

A search box that reads the users table

A05:2025 Injection *drill 1 · TodoController::index(), the to-do search from HW6/HW7*

```
public function index(Request $request) {
    $q = trim((string) $request->query('q', ''));
    $sql = "select id, title, done from todos
            where title like '%$q%' order by title";
    $todos = collect(DB::select($sql));
    $todos = Todo::where('title', 'like', "%$q%")
                 ->orderBy('title')->get();
    return view('todos.index', ['todos' => $todos, 'q' => $q]);
}
```

Green is the fix. ~~Struck through~~ is what it replaces.

THE ATTACK

A visitor types ' UNION SELECT id, email, 0 FROM users WHERE email LIKE '% into the search box. The list fills with every user's email. The same trick reaches password hashes.

WHY THE FIX WORKS

The typed text was pasted into the SQL, so it became SQL. A bound parameter (?) sends the value separately — the database treats it only as text to compare, never as a command. where() does this for you.

THE FIX, IN ONE LINE

Todo::where('title','like',"%$q%")

Same payload, after the fix

PREDICT After the fix, a visitor types the exact same attack into the search box:

```
search box:    ' UNION SELECT id, email, 0 FROM users WHERE email LIKE '%

the bound query Eloquent runs:
select * from todos where title like ? order by title
  ? = "%' UNION SELECT id, email, 0 FROM users WHERE email LIKE '%%"
```

Predict: what does the page show now? (Nothing — it searches for a to-do whose title literally contains that whole string, and finds none. The value is just a value.)

A note that runs in everyone's browser

A05:2025 Injection (XSS) *drill 2 · todos/show.blade.php, a note on a shared to-do*

```
{{-- resources/views/todos/show.blade.php --}}
<h2>Notes</h2>
~~<div class="notes">{!! $todo->notes ?: 'No notes.' !!}</div>~~
<div class="notes">{{ $todo->notes ?: 'No notes.' }}</div>

the note someone left on the shared "Pantry potluck" to-do:
  <script>alert(document.cookie)</script>Sign up here!
```

Green is the fix. ~~Struck through~~ is what it replaces.

THE ATTACK

Whoever last edited the list typed a <script> into a note. {!! !!} prints it as raw HTML, so the browser runs it — their script, in your page, with your cookies.

WHY THE FIX WORKS

{{ }} escapes first: < becomes <, so the browser shows the characters instead of running them. Blade escapes by default; the bug is the one place that asked it not to.

THE FIX, IN ONE LINE

{{ $todo->notes }} not {!! … !!}

Injection, in two lines

A05 · DATABASE · DRILL 1

SQL injection

Never build SQL by pasting in input. Bind the value: where('title','like',$q) sends it separately.

A05 · BROWSER · DRILL 2

Cross-site scripting

Never print untrusted data as raw HTML. {{ }} escapes; {!! !!} does not.

The idea under all of them: the attacker can send anything — so never let what they send be read as code. Bind it, or escape it.

Cross-site request forgery

A01:2025. Your browser attaches your cookie to a request to a site — even when another site is what triggered the request.

Drill 3 on Friday. Your project's React api.js carries the same defense by hand.

A page on another site. Does the server accept it?

PREDICT You use the to-do app in one tab. In another tab you open a page on a different site, which contains only this:

```
<form id="evil" method="POST" action="http://localhost:8000/todos">
    <input type="hidden" name="title" value="Call the attacker: 555-0100">
</form>
<script>document.getElementById('evil').submit();</script>
```

Predict: the server receives a POST it did not ask for, from a page it did not serve. Can it tell? Does it add the to-do? (Hint: what, in the request itself, says which page sent it — and what does a cookie actually prove?)

Another site adds to-dos to your list

A01:2025 Broken Access Control *drill 3 · todos/index.blade.php and bootstrap/app.php*

```
{{-- resources/views/todos/index.blade.php --}}
<form method="POST" action="{{ route('todos.store') }}" class="add">
    @csrf
    <input type="text" name="title" placeholder="New to-do" required>
    <button type="submit">Add</button>
</form>

bootstrap/app.php
~~$middleware->validateCsrfTokens(except: ['todos']);~~
```

Green is the fix. ~~Struck through~~ is what it replaces.

THE ATTACK

A page on another site POSTs to /todos on its own. The server can't tell, from the request alone, which page sent it — and the starter switched the check off for /todos. A to-do you never typed appears.

WHY THE FIX WORKS

@csrf puts a per-session token in the form. The attacker's page can't read it (the browser keeps it from other origins), so it can't copy it. Laravel checks the token on every POST — unless you switch the check off. Two layers you get free in 2026: the session cookie is SameSite=Lax, and Laravel 13 reads the browser's Sec-Fetch-Site header. The token is the layer that works in every browser.

THE FIX, IN ONE LINE

@csrf in the form; delete the except line

CSRF, in two lines

A01 · REQUESTS · DRILL 3

The attack

Another site makes your browser send a request your server treats as yours. From the request alone, the server cannot tell which page sent it.

A01 · TOKEN

The defense

Three layers: a SameSite cookie, the browser's Sec-Fetch-Site header, and a token only your own pages can read. @csrf in Blade; the XSRF-TOKEN header in your React api.js.

The idea under all of them: a cookie proves whose browser it is, not who meant to send the request. The token proves intent, in every browser.

The platform

A02 and A03:2025. Not a line of your code — the secrets you store, the packages you install, and the headers you send (or don't).

Drills 4, 5 and 6 on Friday.

A secret committed in code

A02:2025 Security Misconfiguration *drill 4 · config/pantry.php, the notifications API key*

```
config/pantry.php    (committed)
'api_key' => 'sk_test_00000000000000000000000000',
'api_key' => env('PANTRY_API_KEY'),

.env    (listed in .gitignore, never committed)
PANTRY_API_KEY=sk_test_00000000000000000000000000

.env.example    (committed: the NAME, so teammates know to set it)
PANTRY_API_KEY=
```

Green is the fix. ~~Struck through~~ is what it replaces.

THE ATTACK

The key sits in a file git tracks. Everyone with the repo has it — and deleting it next commit doesn't help: git log -p still shows it. A leaked key must be rotated, not just removed.

WHY THE FIX WORKS

.env is in .gitignore, so it never enters the repository. Code reads secrets with env(), in config/ files only. .env.example records the name so teammates know what to set. The key lives on each machine, not in git.

THE FIX, IN ONE LINE

env('PANTRY_API_KEY') + a line in .env

A dependency with a known hole

A03:2025 Software Supply Chain Failures *drill 5 · composer.json / composer.lock, the packages your app installs*

```
in a second terminal:
$ docker compose exec app composer audit

firebase/php-jwt  5.5.1
  Severity: critical
  Advisory: weak encryption / algorithm confusion

$ docker run --rm -v "$PWD":/app -w /app composer:2 \
    composer remove firebase/php-jwt --no-install --no-scripts
(in use? composer update firebase/php-jwt  moves past the advisory)
```

Green is the fix. ~~Struck through~~ is what it replaces.

THE ATTACK

Your app installs a few hundred packages you never read. One of them, pulled in by one you chose, has a published, critical advisory. You didn't write it, but it ships in your app.

WHY THE FIX WORKS

composer audit (and npm audit) compare every installed version against a public advisory list. When one is flagged, you update past it or remove it. An install script is ordinary code that runs the moment you install.

THE FIX, IN ONE LINE

composer audit -> update or remove

Four headers, four things the browser refuses

A02:2025 Security Misconfiguration *drill 6 · app/Http/Middleware/SecurityHeaders.php and bootstrap/app.php*

Header	Value	Tells the browser to refuse
`X-Content-Type-Options`	`nosniff`	guessing a file's type (MIME sniffing)
`X-Frame-Options`	`DENY`	showing your page in another site's frame (clickjacking)
`Content-Security-Policy`	`default-src 'self'`	loading scripts from anywhere but your origin
`Strict-Transport-Security`	`max-age=31536000; includeSubDomains`	ever dropping back to plain HTTP (forces TLS)

```php
// app/Http/Middleware/SecurityHeaders.php — runs on every response
public function handle(Request $request, Closure $next): Response {
    $response = $next($request);
    $response->headers->set('X-Content-Type-Options', 'nosniff');
    $response->headers->set('X-Frame-Options', 'DENY');
    $response->headers->set('Content-Security-Policy', "default-src 'self'");
    $response->headers->set('Strict-Transport-Security',
        'max-age=31536000; includeSubDomains');
    return $response;
}
// bootstrap/app.php — or it never runs
$middleware->append(\App\Http\Middleware\SecurityHeaders::class);
```

Why it works: a browser left without these headers guesses, and its guesses favour the attacker. Each header is one instruction, set once in a middleware that runs on every response. HSTS is how TLS stops being optional: after the first visit, the browser refuses http:// for your whole domain.

The platform, in three lines

A02 · SECRETS · DRILL 4

Secrets

Keys live in .env (git-ignored), read with env() in config/. A secret in git is leaked, even after you delete it.

A03 · DEPENDENCIES · DRILL 5

Supply chain

You ship code you didn't write. audit it against the advisory list; update or remove what's flagged.

A02 · HEADERS · DRILL 6

Headers & TLS

Send the four headers, and force HTTPS. The browser's defaults favour the attacker; override them.

The idea under all of them: none of these is your application code. Security is the whole system — what you store, what you install, what you send.

Six fixes, three design principles

Saltzer and Schroeder named these principles fifty years ago, before the web existed. Every one of today's fixes is one of them.

Complete mediation

Check every input, every time.

Binding and escaping check every value on the way in and out (drills 1, 2). A policy checked on every write, not once at login. An allow-list of fillable columns. validate() on every form.

Fail-safe defaults

Default to "no". Open on purpose.

CSRF on, headers set, debug off (drills 3, 6). The auth middleware answers 401 before the controller runs. The secure state is the one you don't have to remember.

Least privilege

Give each part only what it needs.

Secrets per machine, not in git (drill 4). A resident may edit their own items; only the RA deletes. A page that only hides buttons. A database user that can't drop tables.

You find each hole, then fix it

One to-do app, six holes, in inclass/wk11/laravel. Each drill shows you the attack first — run it, watch it work — then you make the one change that closes it, and press Check result. Same loop as weeks 7 and 9.

A05 · 8 MIN

1 · SQL injection

A search box that reads the users table. TodoController.

A05 · 6 MIN

2 · XSS

A note that runs in your browser. show.blade.php.

A01 · 7 MIN

3 · CSRF

Another site adds to-dos to your list. The form + bootstrap/app.php.

A02 · 5 MIN

4 · A leaked secret

An API key committed to git. config/pantry.php + .env.

A03 · 6 MIN

5 · A bad dependency

composer audit finds a critical advisory. composer remove.

A02 · 8 MIN

6 · Security headers

The four the browser needs. A middleware.

Build it before Friday (it downloads PHP, MySQL and the packages): `cd inclass/wk11/laravel && docker compose watch` then open http://localhost:8000.

DUE & ASSIGNED

This Friday, 11/6

DUE FRIDAY

Homework 7

Your team's Laravel back end: the table, the model, the seed data, and the five routes from your API.md. Bring it running — Homework 8 is the same six fixes, on it.

Plus: push your copy of Friday's lab to your repo at the end of class.

ASSIGNED FRIDAY

Homework 8 · team

Harden your api/: today's six, on your own project — a headers middleware, the CSRF and cookie settings, no raw SQL from input, no raw HTML in React, no secrets in the repo, the audit clean. Graded by reading your repo and requesting your API. Due Fri 11/13.

Homework 9 (Fri 11/13): the login and the write endpoints.

Readings for the lab (week 11 page, items 4 and 5): the Laravel Blade, Configuration and Sanctum sections, and four MDN security guides — the reference for drills 2, 3, 4 and 6, and for Homework 8.

Assume every input is hostile. Make every default fail closed.

That is the whole of today. The framework does it by default; your job is to notice where someone turned the default off — and on Friday, to turn it back on.